

Sioux casus (1 week) — n8n + model: helpdesk triage & Pareto

Doel

In 1 week maak je een **shadow-mode** oplossing voor een veelvoorkomende toepassing van AI en N8N:

- **Helpdesk tickets/mails** automatisch **categoriseren** (triage)
- periodiek een **Pareto/top-N** rapport maken (+ eenvoudige trend)
- **misroutes** detecteren (“hoort bij ander loket”) en een **concept-antwoord** genereren
- alles **traceerbaar**, **veilig-by-default** en **reproduceerbaar**

Je werkt uitsluitend met de aangeleverde demo-data en config in deze repo.

Invoer (aangeleverd)

Demo-data

Te vinden in `demo-data/`:

- `demo-data/tickets.csv`
- (optioneel) `demo-data/license_requests.csv`
- (optioneel) `demo-data/training_completion.csv`

Config

Te vinden in `config/`:

- `config/categories.json`
- `config/queues.json`

Je mag `demo-data/config` uitbreiden, maar je start met deze input.

Wat je bouwt (shadow mode)

Een n8n-workflow die een batch tickets inleest, per ticket een triage-resultaat produceert, en rapportages wegschrijft.

Shadow mode = je geeft advies/rapportage; je verstuurt geen echte mails naar gebruikers.

n8n bereiken (Educom trainees)

Je n8n draait in je **dev container** (via **npm**, niet als aparte Docker-compose).
Open de editor op:

`https://ai.educom.nu/t/<JOUW_HTTP_POORT>/`

(HTTP-poort = SSH-poort + 1000; staat in je welkomstmail. Start met
`~/start-n8n.sh` — **geen SSH-tunnel**.)

Zie `../docs/trainee/n8n-installatie-en-beheer.md`.

Input inlezen in n8n

Kies 1 van deze routes:

- **Route 1 (aanrader): demo-data via git/HTTP ophalen**
 - Zet `tickets.csv` (en optioneel de andere files) in een git repo als “raw” downloadbare bestanden.
 - n8n nodes:
 - * **HTTP Request (GET)** → haal `tickets.csv` op als file/binary
 - * **Spreadsheet File** → parse CSV “from binary” naar JSON items
 - Voordeel: geen container file-gedoe; alles reproduceerbaar.
 - **Route 2: files in je container plaatsen (SSH)**
 - Log in via SSH en zet `tickets.csv` in je home of workspace (bijv. `~/demo-data/tickets.csv`).
 - n8n nodes:
 - * **Read Binary File** → lees dat pad
 - * **Spreadsheet File** → parse CSV “from binary”
 - Voordeel: werkt zonder git; documenteer het pad in je README.
-

Functionele eisen (minimum)

- **P1 Intake:** n8n leest `tickets.csv` in en normaliseert naar 1 schema.
- **P2 Redaction:** voordat je iets logt/opslagt, redacteer je “PII-achtige” patronen (minimaal e-mail/telefoon/BSN-achtige strings).
- **P3 Triage:** per ticket produceer je minimaal:
 - `category` (exact 1; matcht met `categories.json`)
 - `confidence` (0–1 of low/med/high)
 - `route_to` (optioneel; matcht met `queues.json`)
 - `route_reason` (1–2 zinnen)
- **P4 Concept-antwoord:** alleen voor `route_to != null` een `draft_reply` (kort, beleefd, verwijst naar loket).
- **P5 Pareto:** per week (of per batch) top categorieën met counts en %.

- **P6 Trend/drift (simpel):** vergelijk laatste periode met vorige periode en signaleer stijgers/dalers.
 - **P7 Output:**
 - schrijf `reports/latest.json` (of `.md`) met Pareto + trends
 - schrijf `reports/routed_tickets.json` met tickets die naar ander loket moeten
 - **P8 Observability:**
 - elke run krijgt `run_id`
 - log aantallen: verwerkt, errors, routed, per categorie
-

Niet-functionele eisen (minimum)

- **S1 Data-minimalisatie:** in outputs/logs géén volledige ticket bodies opslaan; alleen korte samenvatting of hash.
 - **S2 Idempotency:** her-run van dezelfde dataset maakt geen dubbele outputs (bijv. overschrijven per run of dedupe op id).
 - **S3 Reproduceerbaarheid:** 1 n8n “Execute workflow” moet de run opleveren met hetzelfde resultaat (binnen redelijke variatie als je model stochastic is; leg dit uit).
 - **S4 Config-gedreven:** categorieën/loketten aanpasbaar via bestanden, niet door node-code te wijzigen.
-

Deliverables (wat je oplevert)

1. `docs/requirements.md` (max 1 pagina)
 - probleemdefinitie (kort)
 - 5–10 FR (in jouw woorden)
 - 5–10 NFR (focus: security/privacy/audit/observability)
 - 10 open vragen voor de klant
 - 5 succesmetrics (hoe meten we dat dit werkt?)
 2. `n8n/workflow-export.json`
 3. `reports/` met voorbeeldoutputs van minimaal 1 run
 4. `README.md` met:
 - hoe run je het
 - hoe lees je input in (welke route gekozen)
 - welke keuzes heb je gemaakt (security/logging/evaluatie)
-

Aanpak in 5 werkdagen (suggestie)

- **Dag 1:** requirements doc + begrip van categorieën/loketten + run output format kiezen.

- **Dag 2:** intake + redaction + triage (rules en/of model call) + outputs wegschrijven.
 - **Dag 3:** Pareto + routed tickets report + run_id + basis logging.
 - **Dag 4:** trend/drift + quality checks (sampling) + idempotency.
 - **Dag 5:** README + demo-scenario + “v2 richting klant” reflectie.
-

Demo (wat je live laat zien)

- Run uitvoeren → toon `run_id`
- 3 tickets: 1 duidelijke categorie, 1 misroute met `draft_reply`, 1 “vage” met lage confidence
- Pareto + trend conclusie (“wat is nu #1 probleem?”)
- Toon redaction (PII-achtige strings komen niet terug in outputs/logs)